Lists & Tuples

Python Mastery — Part 1: Foundations · Week 6

Rathinam Trainers & Consultants

Live on Microsoft Teams · recorded

Recap & today

Where we are in the journey

Last week — Functions & Type Hints

- `def` / `return`, and the full **argument grammar**
- Defaults, `*args`, `**kwargs`, keyword-only, positional-only
- **LEGB** scope, `global` / `nonlocal`, lambdas
- **Type hints**: `def add(a: int, b: int) -> int:`

> You can now package logic into reusable units. Today: what those units **carry**.

Today's goal

By the end of this session you'll understand how to:

- Build and change **lists** with the core methods
- Slice, slice-**assign**, and delete with `del`
- Reason about **mutability** and the **aliasing** trap
- Use **tuples** for fixed records, and **unpack** them fluently

Working with lists

Python's everyday container

What is a list?

```
tasks = ["write report", "buy milk", "call bank"]
```

- **Ordered** — items keep the position you put them in
- **Mutable** — it can be changed *in place*, after it's built
- Can hold **anything**, even mixed types
- `len(tasks)` → `3` · `"buy milk" in tasks` → `True`

Square brackets `[]` make a list. Square brackets also *index* one.

Growing a list

```
>>> tasks.append("call bank")          # one item, at the end
>>> tasks.insert(0, "pay rent")        # one item, at a position
>>> tasks.extend(["water plants", "book flights"])   # many items
```

- `append` adds **one** item — even if that item is a list
- `insert(i, x)` puts `x` **before** index `i`
- `extend` unpacks an iterable and adds **each** item

All three change the list **in place** and return `None`.

Shrinking a list

```
>>> tasks.remove("buy milk")   # by VALUE — first match only
>>> tasks.pop()                # by POSITION — returns the item
'book flights'
>>> tasks.pop(0)               # pop from the front
'pay rent'
>>> tasks.clear()              # empty it
```

- `remove` takes a **value**; `pop` takes an **index**
- `pop` **returns** what it removed — `remove` returns `None`
- `remove` on a missing value → `ValueError`

Finding & counting

```
>>> tasks.index("call bank")
1
>>> tasks.count("call bank")
1
```

- `index(value)` → position of the **first** match, else `ValueError`
- `count(value)` → how many times it appears (`0` is safe)

> `index` accepts optional `start` / `stop` — but **positionally only**: `tasks.index("x", start=0)` raises `TypeError`.

Rathinam Trainers

Reordering & copying

```
>>> nums = [3, 1, 2]
>>> nums.sort()           # in place, returns None
>>> nums.reverse()        # in place, returns None
>>> shallow = nums.copy()
```

- `sort()` and `reverse()` rearrange **this** list and return `None`
- `sort(key=..., reverse=...)` — **keyword-only** arguments
- `copy()` hands back a **new** list — we'll see why that matters

Indexing & slicing

```
>>> tasks[0]        # first
'pay rent'
>>> tasks[-1]       # last
'water plants'
>>> tasks[1:3]      # a NEW list, items 1 and 2
['write report', 'call bank']
```

- Indexing gives you **one item**
- Slicing gives you a **new list** — the original is untouched
- Same `[start:stop:step]` grammar you learned on strings

Slice assignment

```
>>> tasks
['pay rent', 'write report', 'call bank', 'water plants']
>>> tasks[1:3] = ["ring the bank"]
>>> tasks
['pay rent', 'ring the bank', 'water plants']
```

- Assign **to a slice** to replace a whole range at once
- The replacement can be **longer or shorter** — the list resizes
- Strings can't do this. Lists can, because lists are **mutable**.

The `del` statement

```
>>> a = [-1, 1, 66.25, 333, 333, 1234.5]
>>> del a[0]          # [1, 66.25, 333, 333, 1234.5]
>>> del a[2:4]        # [1, 66.25, 1234.5]
>>> del a[:]          # []       – empties it
>>> del a             # the NAME a is gone
```

- `del` removes **by position**, and returns nothing
- `pop` removes *and hands the item back*; `del` just deletes
- `del a[:]` empties the list; `del a` removes the variable itself

Mutation & aliasing

The trap that catches everyone

Mutable means it changes

Strings are **immutable** — methods hand back a *new* string:

```
name.upper()          # returns a new string
```

Lists are **mutable** — methods change the *same* list:

```
tasks.append("x")     # returns None; tasks itself changed
```

> A name doesn't hold a list. A name **points at** one.

Aliasing — two names, one list

```
>>> a = ["write report", "buy milk"]
>>> b = a                  # NOT a copy — a second name
>>> b.append("call bank")
>>> a
['write report', 'buy milk', 'call bank']
```

- `b = a` copies the **arrow**, not the list
- `a is b` → `True` — one single list, two names
- Change it through `b`, and `a` sees it too

The copy fix

```
>>> c = a.copy()      # or  a[:]   or  list(a)
>>> c is a
False
>>> c == a
True
```

- Three ways to copy: `.copy()`, `[:]`, `list(a)`
- `is` asks **same object?** · `==` asks **same contents?**
- This is a **shallow** copy — fine for our tuples of text

Let's mutate a list, live.

Watch this — Demo 11

List mutation & aliasing pitfalls

- Build a to-do list, then `append` / `insert` / `extend` / `remove` / `pop`
- Slice-assign a whole range away
- The **aliasing bug**: `b = a`, then `b.append(...)` changes `a`
- The fix: `b = a.copy()`
- `.sort()` mutating vs `sorted()` returning a new list

The aliasing bug

```
>>> a = ["write report", "buy milk"]
>>> b = a
>>> b.append("call bank")
>>> a
['write report', 'buy milk', 'call bank']
>>> a is b
True
```

Two names. **One** list.

What you just saw

- List methods change the list **in place** and return `None`
- `pop` hands the item back; `remove` matches by value
- `b = a` is an **alias** — not a copy
- `.copy()` / `[:]` / `list(a)` break the link
- `is` compares **identity**; `==` compares **contents**

Tuples

Fixed records that can't wobble

What is a tuple?

```
>>> task = (1, "write report", False)
>>> task[1]
'write report'
>>> task[0] = 9
TypeError: 'tuple' object does not support item assignment
```

- **Ordered** like a list, but **immutable** — no `append`, no `del`
- Use a **list** for "many of the same thing" that grows
- Use a **tuple** for "one record with fixed fields"

The one-element trap

```
>>> type((1))
<class 'int'>
>>> type((1,))
<class 'tuple'>
```

- It's the **comma** that makes a tuple — not the brackets!
- `(1)` is just the number `1` in parentheses
- `1, 2, 3` is a tuple even with **no** brackets at all
- `()` is the empty tuple

Packing & unpacking

```
>>> task = (1, "write report", False)      # packing
>>> tid, title, done = task                # unpacking
>>> title
'write report'
```

- The count must match, or `ValueError`
- Too many → `too many values to unpack (expected 2, got 3)`
- This is why `for tid, title, done in tasks:` works

Star-unpacking & the swap

```
>>> first, *rest = [10, 20, 30, 40]
>>> first, rest
(10, [20, 30, 40])
>>> a, b = b, a          # the swap idiom
```

- `*rest` soaks up **whatever's left** — always a **list**
- Works anywhere: `*most, last` · `head, *mid, tail`
- The swap needs **no temporary variable** — the right side builds first

A list of tuples

```
tasks = [
    (1, "write report", False),
    (2, "buy milk", True),
]

for tid, title, done in tasks:
    print(tid, title, done)
```

- A **mutable** list of **immutable** records — the classic shape
- The list grows and shrinks; each record stays trustworthy
- Unpack right in the `for` — no `task[0]`, `task[1]` noise

Sequence comparison

```
>>> (1, 2, 3) < (1, 2, 4)
True
>>> (1, 2) < (1, 2, 3)
True
```

- Compared **item by item**, left to right — like a dictionary
- First difference decides; if all equal, **shorter is smaller**
- This is exactly *why* `sorted()` can sort a list of tuples

Let's model a task as a tuple.

Watch this — Demo 12

Tuple unpacking & swap

- Model a task as `(id, title, done)` and prove it's immutable
- Unpack it: `tid, title, done = task`
- Swap two names: `a, b = b, a`
- `first, *rest = numbers`
- Iterate a list of task tuples, unpacking in the `for`

Unpacking in a `for`

```
>>> tasks = [(1, "write report", False),
...          (2, "buy milk", True)]
>>> for tid, title, done in tasks:
...     mark = "x" if done else " "
...     print(f"[{mark}] {tid}. {title}")
[ ] 1. write report
[x] 2. buy milk
```

Each tuple unpacks straight into three friendly names.

What you just saw

- A tuple is a **fixed record** — it refuses to be reassigned
- The **comma** makes the tuple, not the brackets
- Unpacking mirrors packing — counts must match
- `*rest` collects the remainder into a **list**
- A **list of tuples** is our first real task model

`.sort()` vs `sorted()`

```
>>> nums = [3, 1, 2]
>>> sorted(nums)        # a NEW list
[1, 2, 3]
>>> nums                # untouched
[3, 1, 2]
>>> nums.sort()         # in place -> returns None
>>> nums
[1, 2, 3]
```

> **The trap:** `nums = nums.sort()` sets `nums` to **`None`**.

Sorting by a key

```
>>> sorted(tasks, key=lambda t: t[1])
[(2, 'buy milk', True), (1, 'write report', False)]
```

- `key=` says **what to sort by** — here, the title
- `.sort()` only works on a **list**; `sorted()` takes **any** sequence
- Mixed types → `TypeError: '<' not supported between ...`

Questions?

Ask me anything about lists & tuples

Wrap-up

This week's homework

Lab 6 — your task list

Build `tasks.py` — an in-memory task list of `(id, title, done)` tuples:

- `add_task` · `remove_task` (by id) · `mark_done` · `sorted_by_title`
- Write them as **typed, documented functions** (Week 5 habit)
- `mark_done` must **replace** the tuple — it can't edit one
- `sorted_by_title` returns a **new** list; stored order stays put

Submit to the **Teams course channel** · `uvx ruff check` clean.

Next week

Week 7 — Sets & Dictionaries

- Sets and set algebra: `|`, `&`, `-`, `^`
- Dictionaries: keyed access, `.items()`, `get`
- Choosing: list vs set vs dict
- `Counter`, `defaultdict`, `namedtuple`

> Your list of tuples becomes a **dict keyed by id**.

See you in the lab.

Bring your task list to Week 7.